

CO5_AT1 Constraint-Based Problem Solving

Name: C. Himaja

Reg. No: 192373035

Source code

```
#include <stdio.h>

#include <limits.h>

#define MAX_CITIES 10

int n;

int cost[MAX_CITIES][MAX_CITIES];

int visited[MAX_CITIES];
int currentPath[MAX_CITIES];
int bestPath[MAX_CITIES];

int bestCost = INT_MAX;

/*
    Find the minimum edge cost in the graph.
    This value is used to calculate a lower bound.
*/

int findMinEdge() {
    int min = INT_MAX;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i != j && cost[i][j] < min) {
```

```

        min = cost[i][j];
    }
}

return min;
}

/*
    Backtracking + Branch and Bound
*/
void tsp(int currentCity, int count, int currentCost, int minEdge) {

    /*
        If all cities have been visited,
        return to the starting city.
    */
    if (count == n) {

        int totalCost =
            currentCost + cost[currentCity][0];

        if (totalCost < bestCost) {

            bestCost = totalCost;

            for (int i = 0; i < n; i++) {
                bestPath[i] = currentPath[i];
            }

```

```

        bestPath[n] = 0;
    }

    return;
}

/*
    Try visiting every unvisited city.
*/
for (int nextCity = 0; nextCity < n; nextCity++) {

    if (!visited[nextCity]) {

        int newCost =
            currentCost + cost[currentCity][nextCity];

        /*
            Branch and Bound:
            Estimate the minimum possible cost
            of completing the remaining tour.
        */
        int remainingCities = n - count;

        int lowerBound =
            newCost + remainingCities * minEdge;

        /*
            Prune the branch if its lower bound
            is already greater than the best solution.
        */

```

```

        if (lowerBound >= bestCost) {
            continue;
        }

        visited[nextCity] = 1;
        currentPath[count] = nextCity;

        tsp(
            nextCity,
            count + 1,
            newCost,
            minEdge
        );

        /*
         * Backtrack
         */
        visited[nextCity] = 0;
    }
}

int main() {

    printf("=====\n");
    printf(" TRAVELING SALESMAN PROBLEM\n");
    printf(" Backtracking + Branch and Bound\n");
    printf("=====\n");

    printf("\nEnter number of cities (maximum %d): ",

```

```
        MAX_CITIES);
scanf("%d", &n);

if (n < 2 || n > MAX_CITIES) {
    printf("Invalid number of cities.\n");
    return 1;
}
```

```
printf("\nEnter the cost matrix:\n");
```

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        scanf("%d", &cost[i][j]);
    }
}
```

```
/*
```

```
    Initialize visited array.
```

```
*/
```

```
for (int i = 0; i < n; i++) {
    visited[i] = 0;
}
```

```
/*
```

```
    Starting city is City 0.
```

```
*/
```

```
visited[0] = 1;
```

```
currentPath[0] = 0;
```

```
/*
```

```

        Find minimum edge cost.
    */

    int minEdge = findMinEdge();

    /*
        Start Backtracking.
    */

    tsp(
        0,
        1,
        0,
        minEdge
    );

    /*
        Display result.
    */

    printf("\n-----\n");
    printf("Optimal Tour:\n");

    for (int i = 0; i <= n; i++) {

        printf("City %d", bestPath[i]);

        if (i < n)
            printf(" -> ");
    }

    printf("\n\nMinimum Travel Cost = %d\n",
        bestCost);

```

```
printf("-----\n");
```

```
printf("\nBranch and Bound pruning was used to");
```

```
printf("\neliminate unnecessarily expensive paths.\n");
```

```
printf("\n===== \n");
```

```
return 0;
```

```
}
```